

CREATE PERFORMANCE TASK · NOTEBOOK

START HERE

AP COMPUTER SCIENCE PRINCIPLES

Create Performance Task Notebook

Plan it · Build it · Test it · Show it · Explain it

ABOUT ME & MY PROJECT

NAME

CLASS PERIOD

PROJECT TITLE

PROGRAMMING LANGUAGE

PARTNER (IF ALLOWED)

STARTED ON

SUBMITTED AS FINAL ON

HOW TO USE THIS NOTEBOOK

Cut each page along the dashed line and glue it into your composition notebook in order. Fill it in as you go — every session. Your notes here are what you'll study before the written responses on exam day.

THE 3 THINGS YOU SUBMIT

1. Your **program code** (as a PDF)
2. A **video** of your program running
3. Your **Personalized Project Reference** (PPR)

THEN, ON EXAM DAY

You answer **written-response questions** about YOUR program, using your PPR. The Create task is **30%** of your AP score.

CREATE PERFORMANCE TASK · NOTEBOOK

BIG PICTURE

1 What Is the Create Performance Task?

Read this whole page before you start. Highlight anything that surprises you.

IN ONE SENTENCE

You design and build your own program, submit it with a short video and a reference sheet of your code, and then explain your code in writing on exam day.

WHAT YOU SUBMIT (THROUGH THE AP DIGITAL PORTFOLIO)

1. **Program code** — all of your code, saved as a PDF. Mark any code you didn't write yourself.
2. **Video** — 1 minute or less showing your program running: input, what it does, and output. No voice narration.
3. **Personalized Project Reference (PPR)** — screenshots of your procedure, where it's called, and your list. No comments.

WHAT HAPPENS ON EXAM DAY

You get a printed copy of your PPR (only your PPR — not your whole program).

You answer written-response questions about:

- your program's purpose and how it works
- your algorithm
- errors and testing
- your list and your procedure

If you don't truly understand your own code, you can't answer them.

KEY DATES

Official in-class Create Task time: **11 sessions = 705 minutes (11.75 hours)**. The College Board requires at least 9 hours.

Deadline to submit as FINAL: **April 30, 2027, 11:59 PM ET** (confirm with your teacher). Your teacher may set an earlier class deadline: _____

MOST IMPORTANT RULE

This must be YOUR work. You must be able to explain every line of your code — what it does and why it's there.

CREATE PERFORMANCE TASK · NOTEBOOK

BIG PICTURE

2

Program Requirements

Your program **MUST** include every item below. Check each one off when your code has it, and write where it is.

YOUR PROGRAM MUST HAVE...

REQUIRED

Requirement	Where it is in my code (line / procedure name)	✓
Input — from the user, a device, an online data stream, or a file		
A list (or other collection) that stores data and is used to fulfill the program's purpose		
A procedure I wrote with a name and at least one parameter that affects what it does		
An algorithm inside that procedure that uses sequencing , selection (IF), and iteration (a loop)		
At least one call to my procedure		
Output (visual, text, sound, or tactile) based on the input and what the program does		

VOCABULARY CHECK — DEFINE IN YOUR OWN WORDS

Parameter:

Argument:

Sequencing / Selection / Iteration:

TIP

The list should be *essential*. If you could replace it with two or three variables and nothing changes, it's not managing complexity.

CREATE PERFORMANCE TASK · NOTEBOOK

BIG PICTURE

3

Rules, Help & Honesty

Know these before you write a single line of code.

WORKING WITH OTHERS

- If your teacher allows it, you may develop the **program code** with a partner.
- Your **video, PPR, and written responses** must be your own.
- Every student submits their own Create task.

YOUR TEACHER CAN...

- explain the task, requirements, and deadlines
 - help you understand programming concepts in general
 - help with technical problems (computer, saving, uploading)
- ...**but can't** write, fix, or give feedback on your Create task code during official time.

USING OUTSIDE CODE OR AI TOOLS

- You may use code from libraries, tutorials, classmates, or AI tools — but you must **acknowledge it** in a comment (who/where it came from).
- The procedure and list you put in your PPR should be **your own work**.
- You must be able to explain ANY code in your program — including code you didn't write.

ACADEMIC INTEGRITY

Copying someone else's program, or submitting work you can't explain, can lead to a score of 0. When in doubt, ask and cite.

I UNDERSTAND THE RULES — SIGN HERE

SIGNATURE _____

DATE _____

CREATE PERFORMANCE TASK · NOTEBOOK

PLAN

4 Timeline & Hours Log

Fill in one row at the end of every official session. Your teacher initials it.

OFFICIAL SESSION LOG

Date	Min	What I finished today	What's next	Teacher
Thu Feb 18	80			
Mon Feb 22	55			
Tue Feb 23	55			
Thu Feb 25	80			
Mon Mar 1	55			
Tue Mar 2	55			
Thu Mar 4	80			
Mon Mar 8	55			
Tue Mar 9	55			
Thu Mar 11	80			
Mon Mar 15	55			

TOTAL MINUTES

Planned: 705 · Actual: _____

MY PERSONAL DEADLINE

Program done by: _____ Video by: _____

NOTES & REMINDERS (MISSED A SESSION? WHAT DO I NEED TO CATCH UP ON?)

CREATE PERFORMANCE TASK · NOTEBOOK

PLAN

5

Brainstorm: Three Ideas

Good projects solve a problem or explore something you care about. Think about school, hobbies, sports, games, music, family, your community.

IDEA 1

Problem it solves / interest it explores:

What list would it need? (what data?)

Who would use it?

What could my procedure do?

IDEA 2

Problem it solves / interest it explores:

What list would it need? (what data?)

Who would use it?

What could my procedure do?

IDEA 3

Problem it solves / interest it explores:

What list would it need? (what data?)

Who would use it?

What could my procedure do?

IDEA STARTERS

quiz or flashcard game · workout or study planner · recipe or meal picker · grade or budget tracker · playlist or movie recommender · trivia about your favorite team · text-adventure game · habit tracker

CREATE PERFORMANCE TASK · NOTEBOOK

PLAN

6 Choose & Check Your Idea

Pick ONE idea and prove it can meet every requirement before you build.

MY CHOSEN IDEA

REQUIREMENT CHECK

REQUIRED

Requirement	How my idea will meet it
Input	
List (what data, why needed)	
Procedure + parameter	
Sequencing, selection, iteration	
Output	

IS IT THE RIGHT SIZE?

- ☐ I can build a working version in the time we have
- ☐ It's not so simple that the list or procedure feels forced
- ☐ I can explain how every part works
- ☐ I have a backup plan if one feature is too hard

WHY I CHOSE THIS IDEA

CREATE PERFORMANCE TASK · NOTEBOOK

DESIGN

7

Purpose, Function, Input & Output

These answers become your written response about program design. Write in complete sentences.

PURPOSE — WHY DOES MY PROGRAM EXIST?

The problem it solves or the creative interest it pursues — not a feature.

FUNCTION — WHAT DOES IT DO?

Describe its behavior from the user's point of view.

INPUT(S) — WHERE THEY COME FROM

OUTPUT(S) — WHAT THE USER SEES, HEARS, OR FEELS

SENTENCE FRAME

When the user _____, the program _____.

WHO ARE MY USERS, AND WHAT DO THEY NEED?

CREATE PERFORMANCE TASK · NOTEBOOK

DESIGN

8

Plan Your List

Your list is your data abstraction. On exam day you'll explain how it manages complexity.

NAME OF MY LIST

REQUIRED

TYPE OF DATA IT STORES

EXAMPLE CONTENTS

Write a few real elements, like ["push-ups", "squats", "plank"]

WHERE DATA GETS STORED IN THE LIST
(HOW IT'S CREATED OR FILLED)

WHERE THE LIST GETS USED (WHAT THE
PROGRAM DOES WITH IT)

HOW DOES THE LIST MANAGE COMPLEXITY?

How would your program have to be written WITHOUT the list? Why would that be harder or impossible?

9 Plan Your Procedure

This is the heart of your project. Plan it before you code it.

PROCEDURE NAME	REQUIRED	PARAMETER(S)	REQUIRED	RETURNS (IF ANYTHING)

WHAT DOES IT DO? (ONE OR TWO SENTENCES)

STEPS OF THE ALGORITHM IN PLAIN ENGLISH

Number them. Mark which step is sequencing (S), selection (SEL), and iteration (IT).

TWO CALLS THAT DO DIFFERENT THINGS

Call (with arguments)	What happens / what it returns

HOW DOES THE PROCEDURE MANAGE COMPLEXITY? (WHAT IF IT DIDN'T EXIST, OR HAD NO PARAMETER?)

10

Storyboard Your Screens

Sketch what the user sees, in order. Label buttons, text boxes, and outputs.

Screen 1: _____

Screen 2: _____

Screen 3: _____

Screen 4: _____

THINK LIKE A USER

Where does the user give input? What changes after they do? What output proves the program worked?

11

Write It in Pseudocode First

Draft your procedure in AP pseudocode (or your language). Planning on paper saves hours of debugging.

AP PSEUDOCODE REMINDERS

```
x ← value · IF (cond) {{ }} ELSE {{ }} · REPEAT n TIMES · FOR EACH item IN list · APPEND(list,
value) · RETURN (value) · lists start at index 1
```

PROCEDURE _____ (_____)

This image shows a single page of white paper with horizontal blue lines, resembling notebook paper. The lines are evenly spaced and run across the width of the page. There is no handwriting or other markings on the paper.

WHERE WILL I CALL IT, AND WITH WHAT ARGUMENTS?

12 **Build Log · Sessions 1–2**

Write a quick entry at the end of every session. Be specific — future you will thank you.

SESSION 1 · DATE: _____

My goal today:

Problems I hit & how I handled them:

What I got done:

Next time I will:

SESSION 2 · DATE: _____

My goal today:

Problems I hit & how I handled them:

What I got done:

Next time I will:

13 **Build Log · Sessions 3–4**

Write a quick entry at the end of every session. Be specific — future you will thank you.

SESSION 3 · DATE: _____	
My goal today:	Problems I hit & how I handled them:
_____	_____
_____	_____
What I got done:	_____
_____	_____
_____	Next time I will:
_____	_____
_____	_____
_____	_____

SESSION 4 · DATE: _____	
My goal today:	Problems I hit & how I handled them:
_____	_____
_____	_____
What I got done:	_____
_____	_____
_____	Next time I will:
_____	_____
_____	_____
_____	_____

14 **Build Log · Sessions 5–6**

Write a quick entry at the end of every session. Be specific — future you will thank you.

SESSION 5 · DATE: _____

My goal today:

Problems I hit & how I handled them:

What I got done:

Next time I will:

SESSION 6 · DATE: _____

My goal today:

Problems I hit & how I handled them:

What I got done:

Next time I will:

15 **Build Log · Sessions 7–8**

Write a quick entry at the end of every session. Be specific — future you will thank you.

SESSION 7 · DATE: _____	
My goal today:	Problems I hit & how I handled them:
_____	_____
_____	_____
_____	_____
What I got done:	
_____	_____
_____	_____
_____	Next time I will:
_____	_____
_____	_____
_____	_____

SESSION 8 · DATE: _____	
My goal today:	Problems I hit & how I handled them:
_____	_____
_____	_____
_____	_____
What I got done:	
_____	_____
_____	_____
_____	Next time I will:
_____	_____
_____	_____
_____	_____

16 **Build Log · Sessions 9–10**

Write a quick entry at the end of every session. Be specific — future you will thank you.

SESSION 9 · DATE: _____	
My goal today:	Problems I hit & how I handled them:
_____	_____
_____	_____
What I got done:	_____
_____	_____
_____	Next time I will:
_____	_____
_____	_____
_____	_____

SESSION 10 · DATE: _____	
My goal today:	Problems I hit & how I handled them:
_____	_____
_____	_____
What I got done:	_____
_____	_____
_____	Next time I will:
_____	_____
_____	_____
_____	_____

17

Build Log · Sessions 11–12

Write a quick entry at the end of every session. Be specific — future you will thank you.

SESSION 11 · DATE: _____

My goal today:	Problems I hit & how I handled them:
_____	_____
_____	_____
_____	_____
What I got done:	
_____	_____
_____	_____
_____	Next time I will:
_____	_____
_____	_____
_____	_____

SESSION 12 · DATE: _____

My goal today:	Problems I hit & how I handled them:
_____	_____
_____	_____
_____	_____
What I got done:	
_____	_____
_____	_____
_____	Next time I will:
_____	_____
_____	_____
_____	_____

18

Testing Log

Test with normal values, boundary values, and weird values. Your written responses will ask about testing.

TEST CASES

#	Input / call	What I expect	What actually happened	Pass?
1				
2				
3				
4				
5				
6				
7				
8				

TWO CALLS TO MY PROCEDURE THAT RUN DIFFERENT PARTS OF THE CODE

REQUIRED

Call (arguments)	Condition being tested	Result
1.		
2.		

CREATE PERFORMANCE TASK · NOTEBOOK

TEST

19

Bug Journal

Every programmer has bugs. Record yours — explaining an error you fixed is great practice for exam day.

BUG 1

What went wrong (what I saw):

How I found it:

Error type (syntax / run-time / logic / overflow):

How I fixed it:

BUG 2

What went wrong (what I saw):

How I found it:

Error type (syntax / run-time / logic / overflow):

How I fixed it:

BUG 3

What went wrong (what I saw):

How I found it:

Error type (syntax / run-time / logic / overflow):

How I fixed it:

DEBUGGING STRATEGIES

test cases · hand tracing · adding print/DISPLAY statements · a debugger · reading error messages carefully · explaining your code out loud to a classmate

CREATE PERFORMANCE TASK · NOTEBOOK

VIDEO

20

Video Requirements & Plan

Your video proves your program runs. Plan it like a movie so you only have to record it once or twice.

YOUR VIDEO MUST

REQUIRED

- ☐ Show your program **running**
- ☐ Show **input** being given to the program
- ☐ Show at least one **feature (functionality)** working
- ☐ Show the **output**
- ☐ Be **1 minute or less**
- ☐ Use an accepted file type (.mp4, .mov, .wmv, .avi, or .webm) and be under the size limit (30 MB)

YOUR VIDEO MUST NOT

- ☐ Include **voice narration** (music is OK; text captions are OK)
- ☐ Show your **name**, face, or any personal information
- ☐ Show your code in a way that replaces showing the program running

RECORDING PLAN

RECORDING TOOL

TEST INPUTS TO SHOW

PRACTICE RUN DONE?

TIPS

Close other tabs and notifications. Make the window big. Move slowly so viewers can see what you type and click. Show input → result clearly, then maybe a second example.

21

Video Script & Shot List

Plan every second. Captions are optional, but they help viewers follow.

SHOT LIST (60 SECONDS MAX)

REQUIRED

Time	What's on screen	What the user does (input)	Caption (optional)
0:00–0:05			
0:05–0:15			
0:15–0:25			
0:25–0:35			
0:35–0:45			
0:45–0:55			
0:55–1:00			

FINAL CHECK AFTER RECORDING

- ☐ Length: _____ seconds (60 or less)
- ☐ Input, functionality, and output are all clearly visible
- ☐ No voice, no name, no personal info
- ☐ File plays correctly and is the right type and size

NOTES FROM MY PRACTICE RECORDING (WHAT TO FIX BEFORE THE FINAL TAKE)

CREATE PERFORMANCE TASK · NOTEBOOK

PPR

22

Personalized Project Reference (PPR)

Your PPR is the **ONLY** part of your code you'll see on exam day. Build it carefully.

YOUR PPR HAS 4 SCREENSHOTS

REQUIRED

Procedure

- i. The whole procedure definition (name, parameter, algorithm)
- ii. Where the procedure is **called**

List

- i. Where data is **stored** in the list
- ii. Where the list is **used** (the same list)

PROCEDURE SCREENSHOT: FROM LINE ___ TO ___ — NOTES

PROCEDURE CALL SCREENSHOT: LINE ___ — NOTES

LIST STORED SCREENSHOT: LINE ___ — NOTES

LIST USED SCREENSHOT: LINE ___ — NOTES

PPR CHECKLIST

- ☐ **No comments** anywhere in the PPR (remove or crop them out)
- ☐ Code is large and readable when printed
- ☐ No names or personal info
- ☐ The list in both list screenshots is the same list
- ☐ Code is readable even if it's split across multiple screenshots

HOW I'LL CAPTURE MY SCREENSHOTS (TOOL, ZOOM LEVEL, WHAT TO CROP OUT)

23

Code Credits & Final Checklist

Give credit for anything you didn't write, then submit everything.

ACKNOWLEDGMENTS (CODE OR IDEAS FROM OTHERS, LIBRARIES, TUTORIALS, AI TOOLS)

What I used	Where / who it came from	Where it is in my code

BEFORE SUBMITTING

REQUIRED

- ☐ Every outside code segment has a comment giving credit
- ☐ My program runs with no errors
- ☐ Code saved as a PDF (all code, readable)
- ☐ Video uploaded and plays correctly
- ☐ PPR uploaded (4 screenshots, no comments)
- ☐ All three components submitted in the AP Digital Portfolio
- ☐ Submission marked **FINAL** by the deadline
- ☐ I saved a copy of my PPR and my code for myself

PROOF OF SUBMISSION

DATE & TIME SUBMITTED AS FINAL _____

TEACHER INITIALS _____

CREATE PERFORMANCE TASK · NOTEBOOK

EXAM PREP

24

Written-Response Practice

Answer using ONLY your PPR, like on exam day. Time yourself: aim for about 5–8 minutes per prompt.

PRACTICE 1 · PROGRAM DESIGN, FUNCTION & PURPOSE

Identify the purpose of your program and the users it's designed for. Describe one input and one output, and how the input affects the output.

PRACTICE 2 · ALGORITHM DEVELOPMENT

Describe how your procedure's algorithm works in enough detail that someone else could recreate it. Point out the sequencing, selection, and iteration.

25

Written-Response Practice

Answer using ONLY your PPR, like on exam day. Time yourself: aim for about 5–8 minutes per prompt.

PRACTICE 3 · ERRORS & TESTING

Describe two calls to your procedure with different arguments that cause different parts of the code to run. For each, describe the condition being tested and the result.

PRACTICE 4 · DATA ABSTRACTION

Explain how your list manages complexity in your program. Explain how your program would need to be written differently without the list.

26

Written-Response Practice

Answer using ONLY your PPR, like on exam day. Time yourself: aim for about 5–8 minutes per prompt.

PRACTICE 5 · PROCEDURAL ABSTRACTION

Explain how your procedure manages complexity. What would you have to do differently if the procedure didn't exist, or if it had no parameter?

PRACTICE 6 · ERRORS & TESTING (BUG)

Describe one error you found while developing your program. Explain how you found it and how you fixed it.

27 **Written-Response Practice**

Answer using ONLY your PPR, like on exam day. Time yourself: aim for about 5–8 minutes per prompt.

PRACTICE 7 · MODIFYING YOUR PROGRAM

Suppose you want to add a new feature. Describe how you would change your procedure or list to do it, and what new testing you would need.

PRACTICE 8 · DIFFERENT INPUT

Choose a different input than the one in your video. Explain step by step how your program would process it and what output it would produce.

28 My Exam-Day Cheat Sheet

Memorize this page. You can't bring it into the exam, but knowing it cold makes the written responses easy.

PROGRAM PURPOSE

MAIN INPUT

MAIN OUTPUT

LIST NAME

WHAT THE LIST STORES

PROCEDURE NAME

PARAMETER(S)

WHAT IT RETURNS / DOES

SEQUENCING IN MY PROCEDURE (LINES / STEPS)

SELECTION IN MY PROCEDURE

ITERATION IN MY PROCEDURE

TWO TEST CALLS I CAN DESCRIBE

THINGS I TEND TO FORGET OR MIX UP

CREATE PERFORMANCE TASK · NOTEBOOK

REFLECT

29

Reflection

Be honest. Thinking about how you worked makes you a better programmer.

WHAT I'M MOST PROUD OF IN MY PROGRAM

THE HARDEST PART WAS... AND HERE'S HOW I GOT THROUGH IT

SOMETHING NEW I LEARNED ABOUT PROGRAMMING

RATE YOURSELF

Planning before coding	①	②	③	④	⑤
Using my time in class	①	②	③	④	⑤
Testing and debugging	①	②	③	④	⑤
Understanding every line of my code	①	②	③	④	⑤
Asking for help when stuck	①	②	③	④	⑤

IF I HAD MORE TIME, I WOULD ADD OR CHANGE...

CREATE PERFORMANCE TASK · NOTEBOOK

REFLECT

30

Suggestions for Next Time

Look back at your build log and bug journal before you answer.

WHAT I WOULD DO DIFFERENTLY ON MY NEXT PROJECT

ADVICE I'D GIVE A STUDENT STARTING THE CREATE TASK NEXT YEAR

ONE HABIT I'LL KEEP (PLANNING, TESTING, COMMENTS, SAVING OFTEN...)

FEEDBACK FOR MY TEACHER: WHAT HELPED ME MOST, AND WHAT WOULD HELP MORE
